



雲端運算服務

Cloud Computing Service

單元05-1 Docker 容器技術

蘇維宗 (Wei-Tsung Su)

suwt@scu.edu.tw

H307-3





Revision

Rev.	Description	Date	Authors
v1.0	Baseline	2024/3/1	蘇維宗

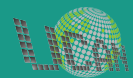




單元內容

- Docker 是什麼?
- 安裝 Docker 環境
- 使用 Docker Desktop
- 使用 Docker CLI
- 製作 Docker 映像檔
- 分享 Docker 映像檔

Docker 是什麼？

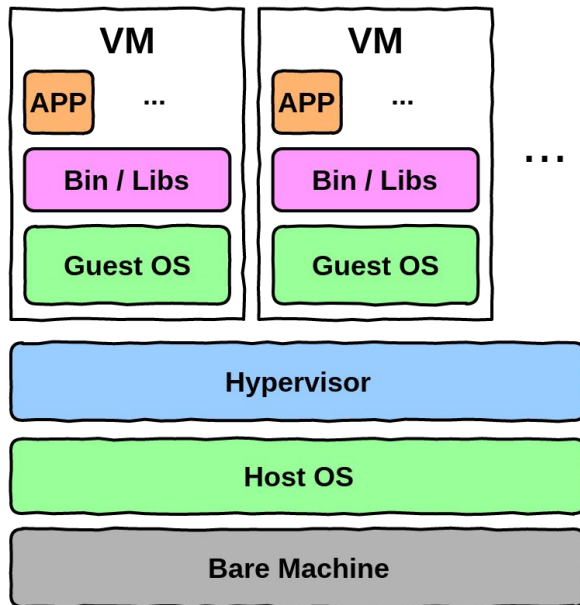




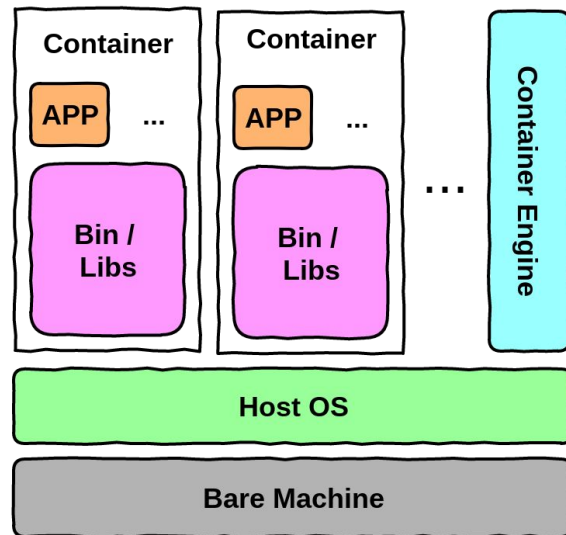
Docker 是什麼？

- Docker是一個虛擬化技術平台，有別於VirtualBox以虛擬機(VM)為基礎的系統層(system-level)虛擬化技術，Docker是基於容器(container)的作業系統層(OS-level)虛擬化技術。
- Container只包含應用程式與所需的執行環境(不含作業系統)是一種輕量級虛擬化技術，這表示container的佈署或遷移會比VM更快速。
- 不過與VM相比，container也有幾個缺點
 - 需要額外且複雜的管理層
 - 共用作業系統核心導致的安全性問題

VM vs. Contrainer



Container 有獨立的執行環境與函式庫，但是因為**共用 Host OS 的核心**，所以在啟動與遷移時會比 VM 快速許多。



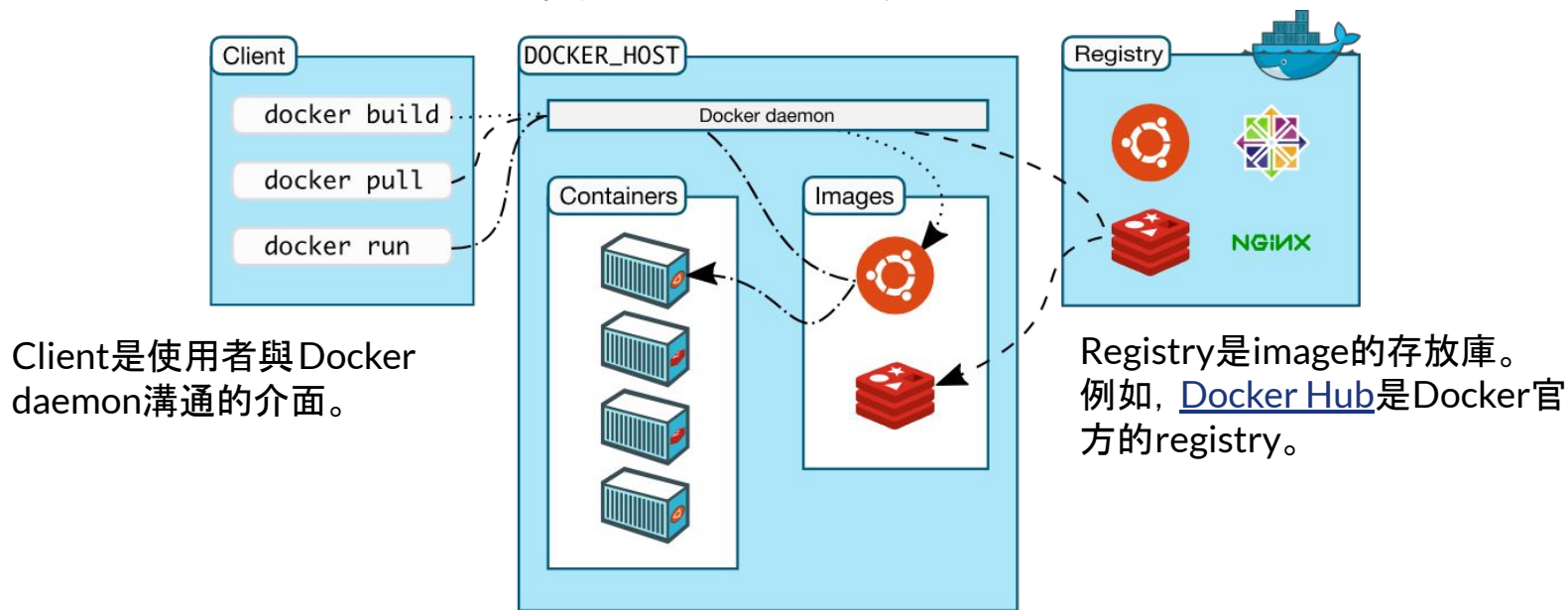


Docker基本概念

- **Image (容器映像)**
 - Image是container的模板。
 - 例如, 在 Docker Hub 上的 [nginx](#) image 為 Nginx 網頁伺服器的模板。
- **Container (容器)**
 - Container是image的執行實體。
 - 例如, 基於 nginx image 建立的 container 能執行 Nginx 網頁伺服器服務。
- **Registry (容器登錄)**
 - Registry是image的存放庫(repository)。
 - 例如, [Docker Hub](#) 是 Docker 官方的 registry。

Docker架構

Docker Host是執行Docker的主機，也是基於image建立並執行container的地方。

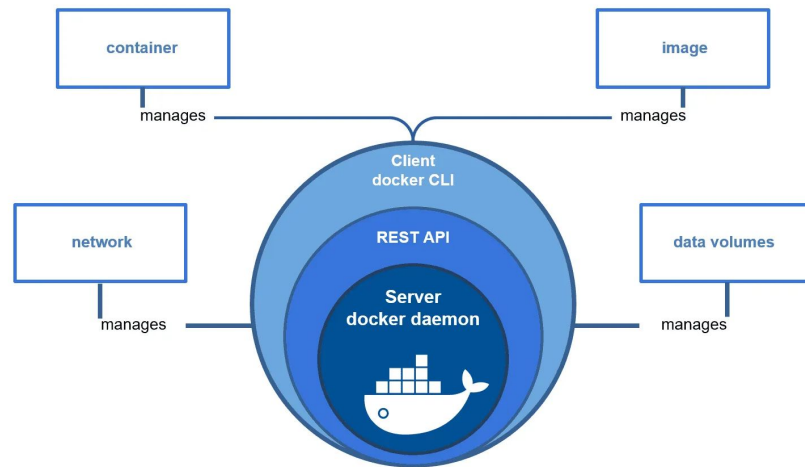


圖片來源：<https://docs.docker.com/get-started/overview/>

Docker Engine

Docker Engine是一個基於client-server架構的應用程式(如右圖), 包含以下三個部分

- Docker daemon (Server)
管理主機上 image與container的服務
- [REST API](#) (Docker Engine API)
可以API/SDK與Docker daemon溝通
- [Docker CLI](#) (Client)
透過指令與Docker daemon溝通



圖片來源

: <https://medium.com/@senali/what-is-the-difference-between-docker-runtime-and-docker-engine-235b25ae9a73>



Docker Image

Image是用來產生container的模板，其來源有

- 官方registry ([Docker Hub](#))
- 非官方registry (例如, [Amazon ECR](#)、[Azure Container Registry](#)、[Google Artifact Registry](#)等)

Image URI是image的唯一識別碼，包含四個部分

- **來源(source)**: 指定image所在的registry，預設為Docker Hub (即docker.io)。
- **帳號(account)**: 指定registry底下的使用者帳號。(部分官方認證的image可以不指定)。
- **存放庫(repo)**: 指定image的名稱。
- **標籤(tag)**: 指定image的標籤(通常代表版本)，預設標籤為latest。





Docker Image URI

Image URI的描述格式為

`[source/] [account/] repo[:tag]`

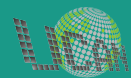
例如, 以docker.io/bitnami/python:latest 為例, 其

- 來源(source): `docker.io` (預設值, 即Docker Hub)
- 帳號(account): `bitnami`
- 存放庫(repo): `python`
- 標籤(tag): `latest` (預設值)

因為來源與標籤都是預設值, 所以也可以簡化成 `bitnami/python`



安裝 Docker 環境



Docker 主要產品

Docker Desktop

在主機上安裝使用
Docker 的環境。

<https://www.docker.com/products/docker-desktop/>



Docker Desktop

Developer productivity tools
and a local Kubernetes
environment.

Download for
Windows



Docker Hub

Cloud-based application
registry and development
team collaboration services.

Signup

Docker Hub

提供存放 Image 的服務。許多官方 Image
會存放在此，也可以
將自己產生的 Image
存放於此。

<https://hub.docker.com/>



Docker 帳號

<https://www.docker.com/pricing/>

Personal

For new developers and/or students getting started with containers.

\$0

- ✓ Docker Desktop
- ✓ Unlimited public repositories
- ✓ 200 image pulls per 6 hours
- ✓ Docker Engine + Kubernetes
- ✓ 3 Scout enabled repos
- ✓ Local Scout analysis

Get Started

Pro

Yearly

For professional developers who want to accelerate innovation.

\$5 per month

Everything in Personal plus:

Docker Desktop Pro

- ✓ Unlimited private repositories
- ✓ 5,000 image pulls per day
- ✓ 5 concurrent builds
- ✓ Synchronized File Shares
- ✓ Docker Debug
- ✓ Local Scout analysis and remediation
- ✓ 5 day support response

Buy now

Team

Yearly

For development teams looking to increase collaboration and agility.

\$9 per user* per month

Everything in Pro plus:

Docker Desktop Teams

- ✓ Up to 100 users
- ✓ Unlimited teams
- ✓ 15 concurrent builds
- ✓ Add users in bulk
- ✓ Audit logs
- ✓ Role based access control
- ✓ 2 day support response

buildcloud

Included minutes: 400/org/month
Included cache: 100/GiB/org/month

Buy now

Business

Yearly

For businesses seeking to establish an enterprise-grade development approach.

\$24 per user* per month

Everything in Team plus:

Docker Desktop Business

- ✓ Hardened Docker Desktop
- ✓ Single Sign-On (SSO)
- ✓ SCIM user provisioning
- ✓ VDI support
- ✓ Private Extensions Marketplace
- ✓ Image and Registry Access Management
- ✓ Purchase via invoice
- ✓ Priority case routing
- ✓ Proactive case monitoring
- ✓ 24-hour support response

buildcloud

Included minutes: 800/org/month
Included cache: 200/GiB/org/month

Contact Sales

Buy now



Hands-on Lab #1

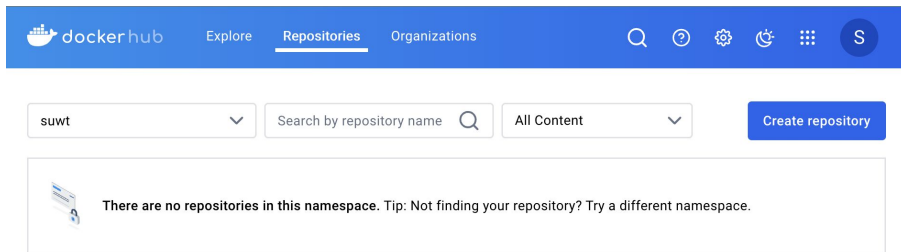
註冊 Docker 帳號

難度: 簡易

時間: 10分鐘

練習目標

在 Docker Hub 上完成 Docker 帳號註冊
<https://hub.docker.com/>





安裝 Docker Desktop

直接到官方網站下載應用程式

<https://www.docker.com/products/docker-desktop>

- Linux 與 Mac (Apple Silicon 或 Intel Chip)作業系統可以直接安裝
- Windows (AMD64 或 ARM64)作業系統(需依據以下兩種不同的 Linux 核心進行安裝)

<https://docs.docker.com/docker-for-windows/install/>

1. (建議使用)預設以Windows Subsystem for Linux 2 (WSL 2)為核心
2. 以Hyper-V為核心(需要Windows專業版/企業版才能使用)



Windows Subsystem for Linux 2 (WSL 2)

WSL 是什麼？

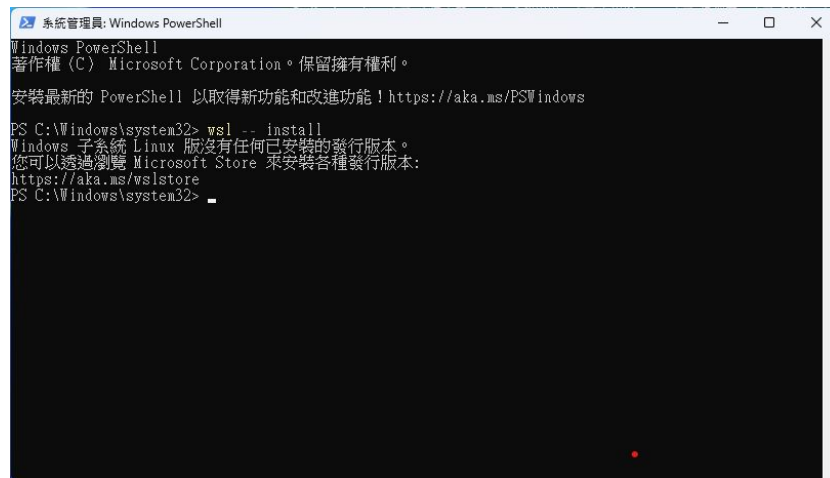
WSL 可以讓使用者在 Windows 電腦上執行 Linux 環境，而無需使用單獨的虛擬機器或雙重開機系統。

WSL 的設計目的是為了給想要同時使用 Windows 和 Linux 的開發人員順暢且具生產力的體驗。

如何安裝 WSL？

以系統管理員身份執行 PowerShell 並輸入以下指令

```
$ wsl --install
```



```
系統管理員: Windows PowerShell
Windows PowerShell
著作權 (C) Microsoft Corporation。保留擁有權利。

安裝最新的 PowerShell 以取得新功能和改進功能！https://aka.ms/PSWindows

PS C:\Windows\system32> wsl -- install
Windows 子系統 Linux 版沒有任何已安裝的發行版本。
您可以透過瀏覽 Microsoft Store 來安裝各種發行版本:
https://aka.ms/wslstore
PS C:\Windows\system32>
```

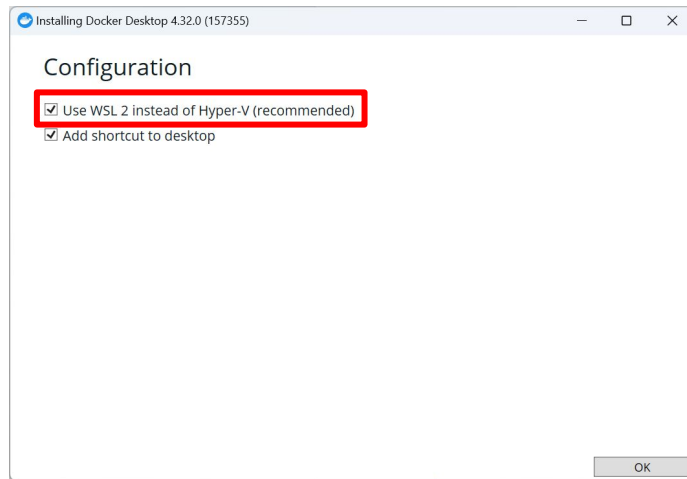
在Windows上安裝 Docker Desktop

在 Windows 上安裝 Docker Desktop 時會出現 Configuration 選單。

預設會勾選使用WSL 2, 請保留此預設選項。

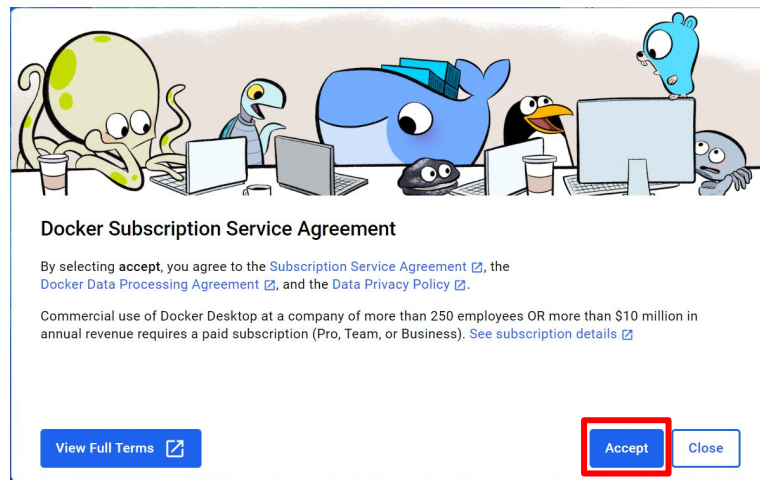
注意

1. 請確定在 Windows 上安裝 WSL 2。
2. 安裝完成後需要重新開機。



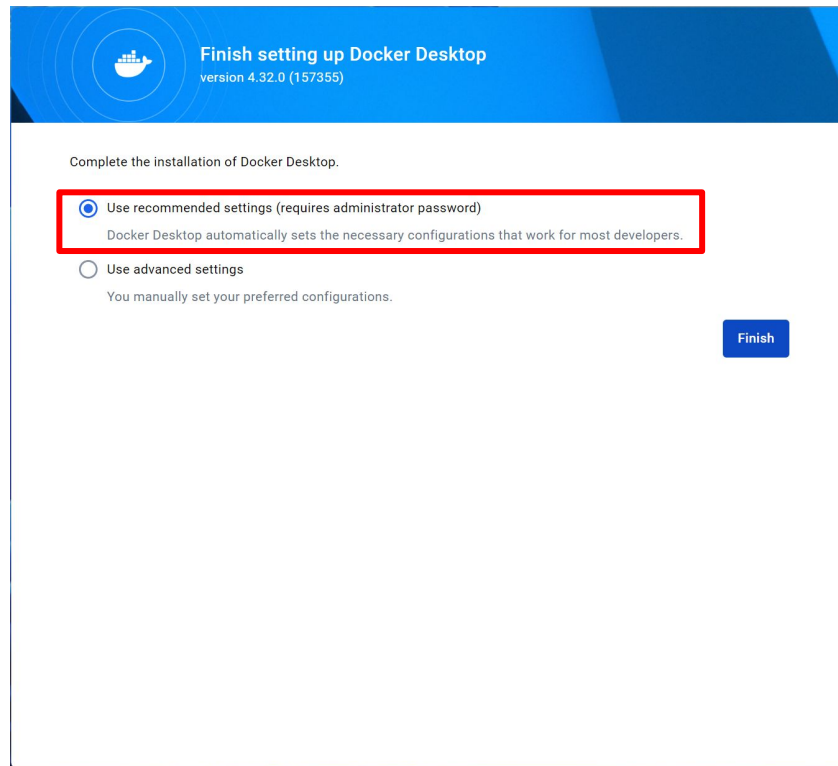
訂閱服務授權

第一次使用 Docker Desktop 時，需要同意訂閱服務授權。



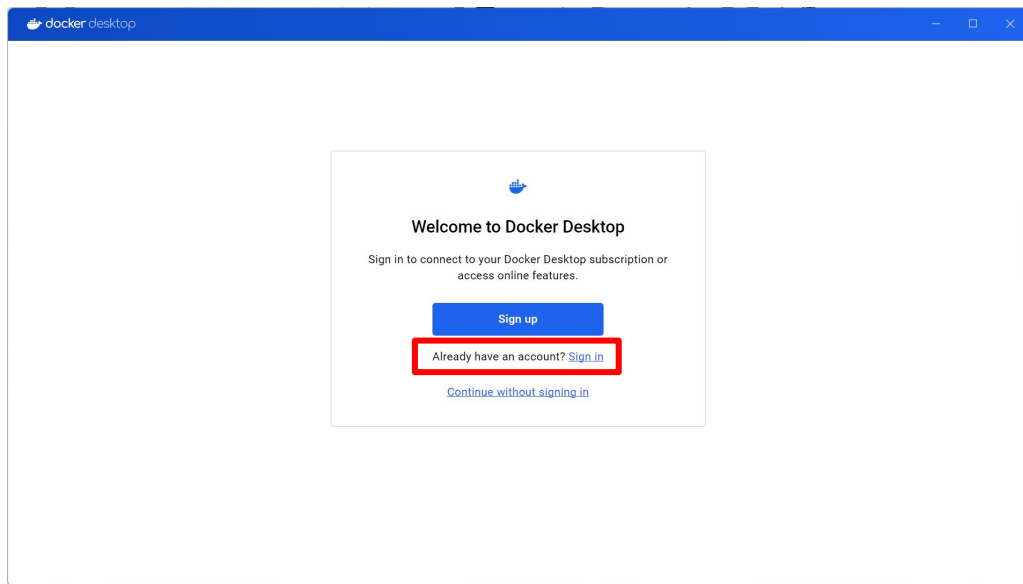
完成設定

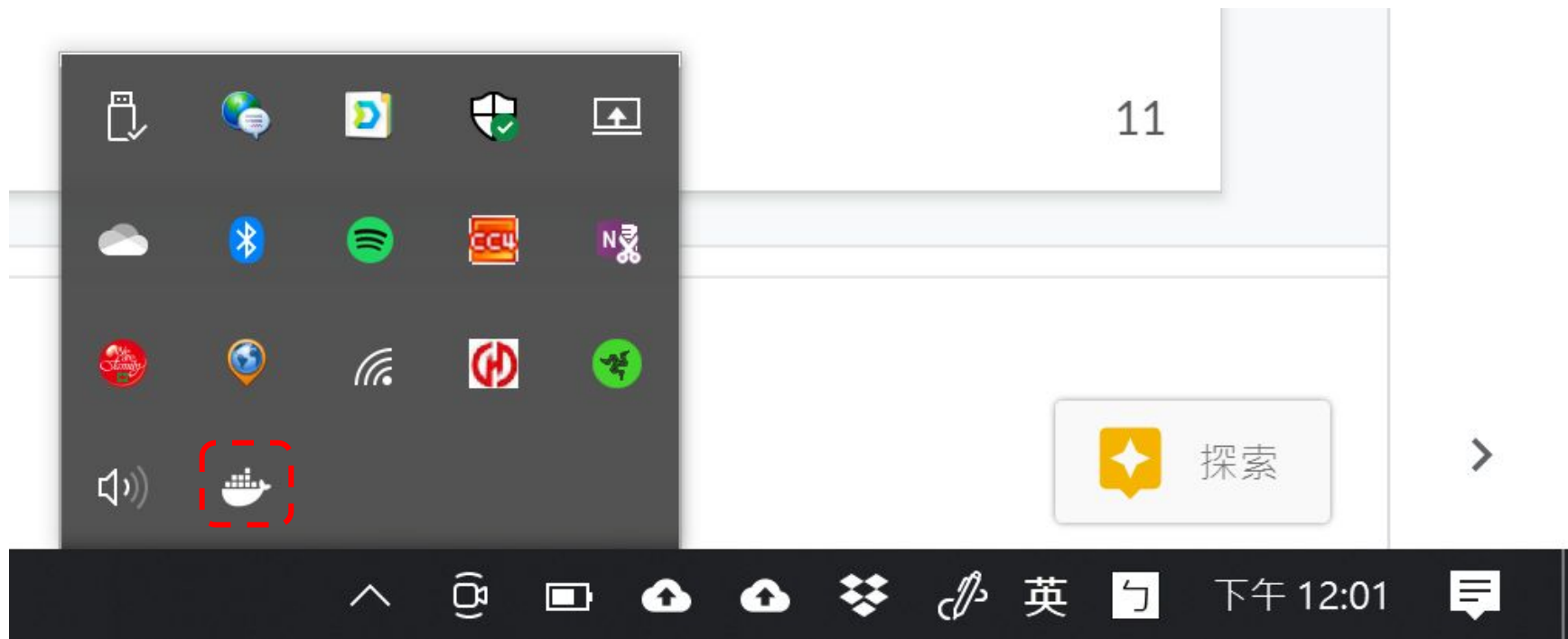
完成 Docker Desktop 設定。
使用建議的設定即可。



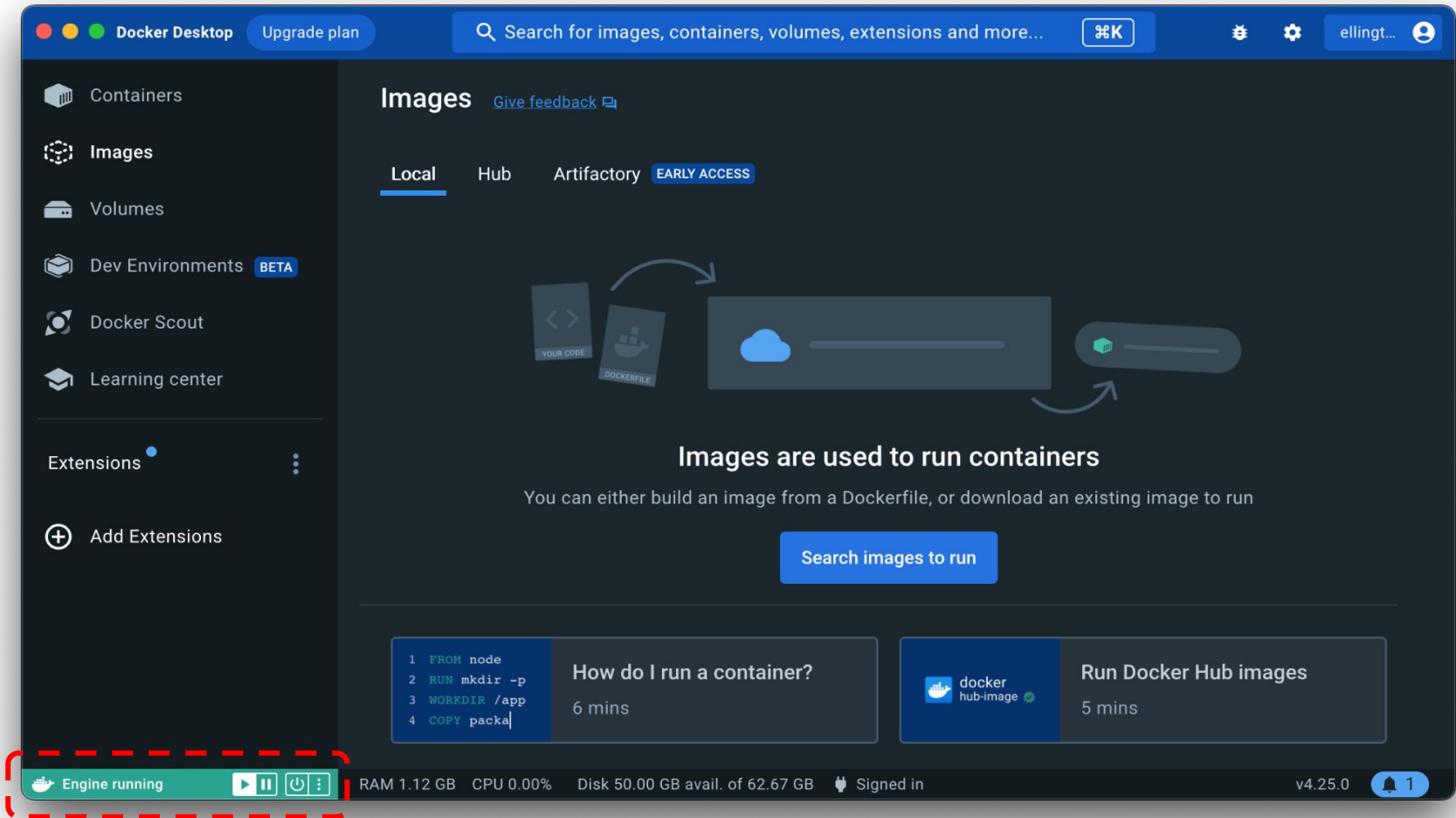
登入 Docker 帳號

使用剛剛在 Docker Hub 上註冊的 Docker 帳號登入 Docker Desktop。



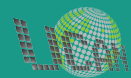


安裝成功可以在快捷列看到 Docker 圖示



使用 Docker Desktop

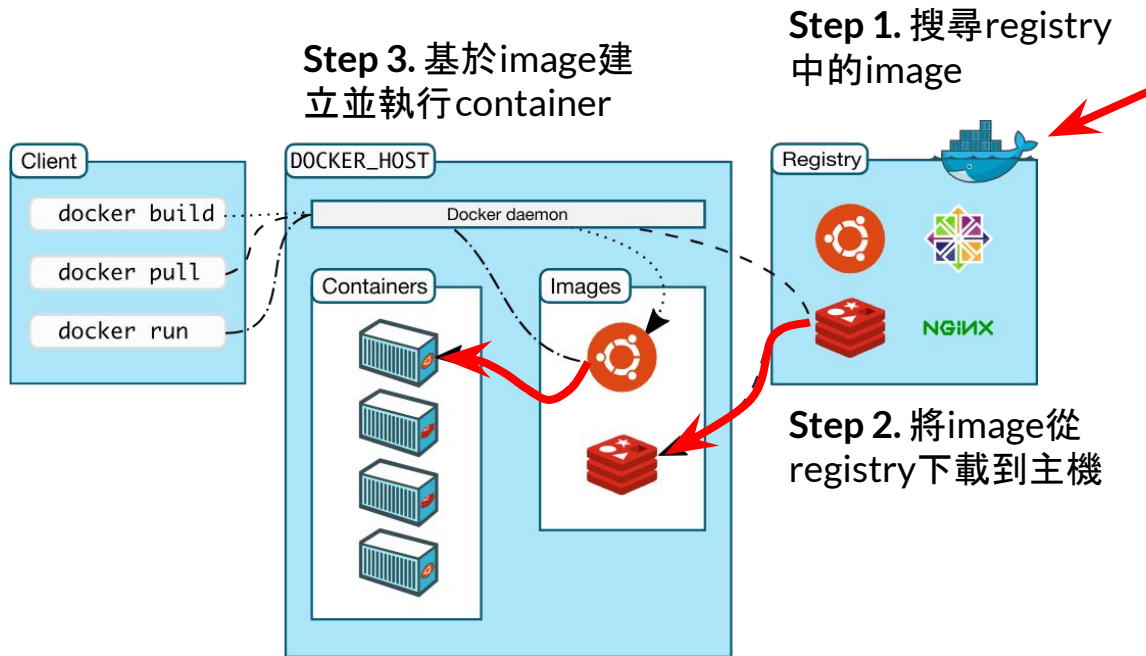
建立與刪除執行環境



建立執行環境

建立執行環境基本步驟

1. 搜尋image
2. 下載image
3. 建立container



圖片來源

: <https://docs.docker.com/get-started/overview/>

刪除執行環境

刪除執行環境基本步驟

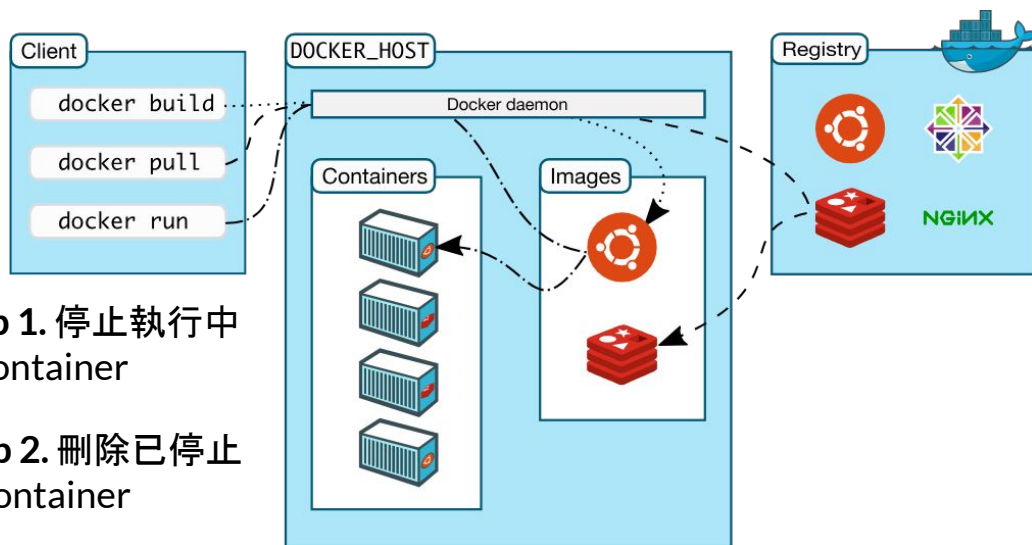
1. 停止container
2. 刪除container
3. 刪除image

Step 1. 停止執行中的container

Step 2. 刪除已停止的container

Step 3. 刪除image

注意! 如果有基於該image所產生的container存在就無法刪除

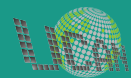


圖片來源

: <https://docs.docker.com/get-started/overview/>

使用 Docker Desktop

使用案例



使用案例 #1: Nginx 執行環境

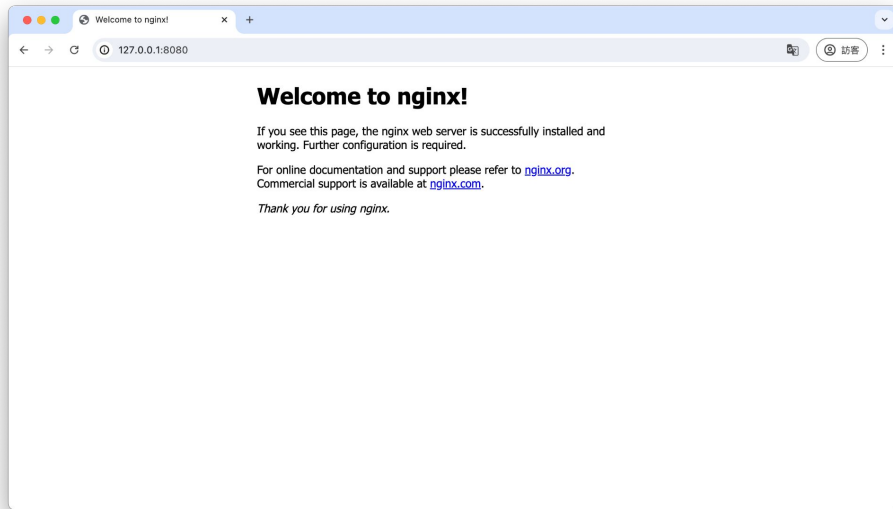
需求情境

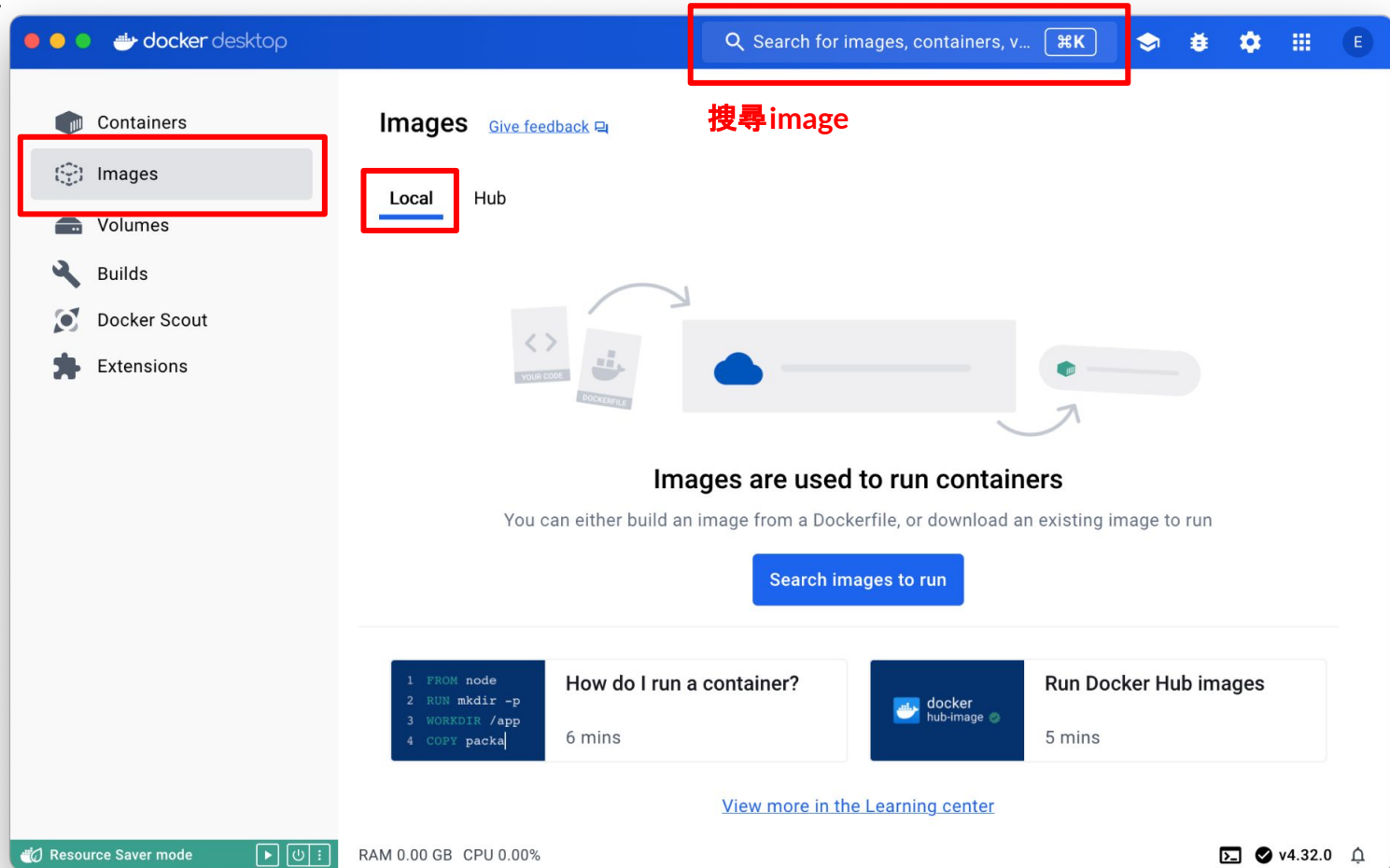
專題(或作業)需要在電腦主機上架設一個網頁伺服器。

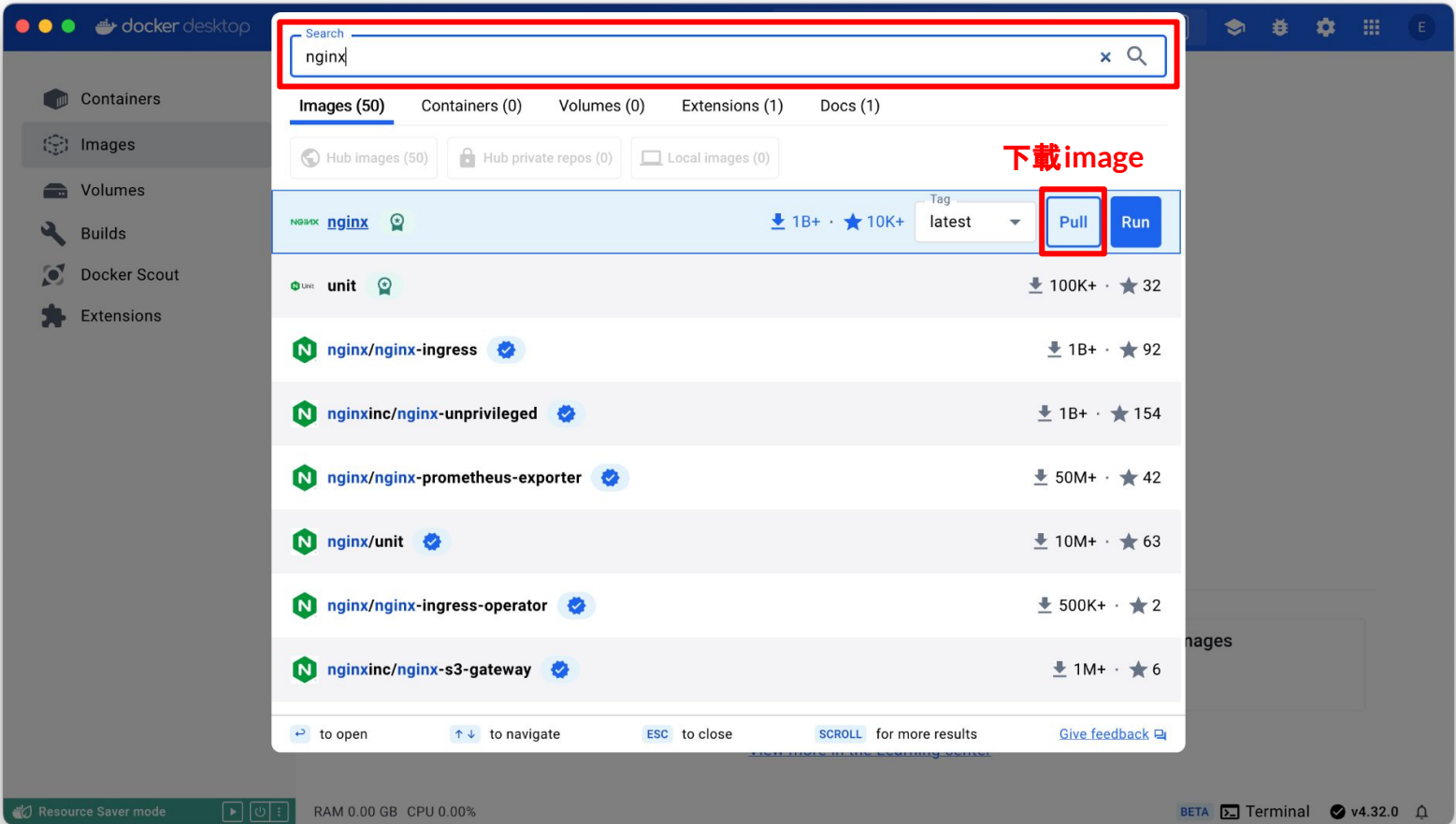
操作步驟

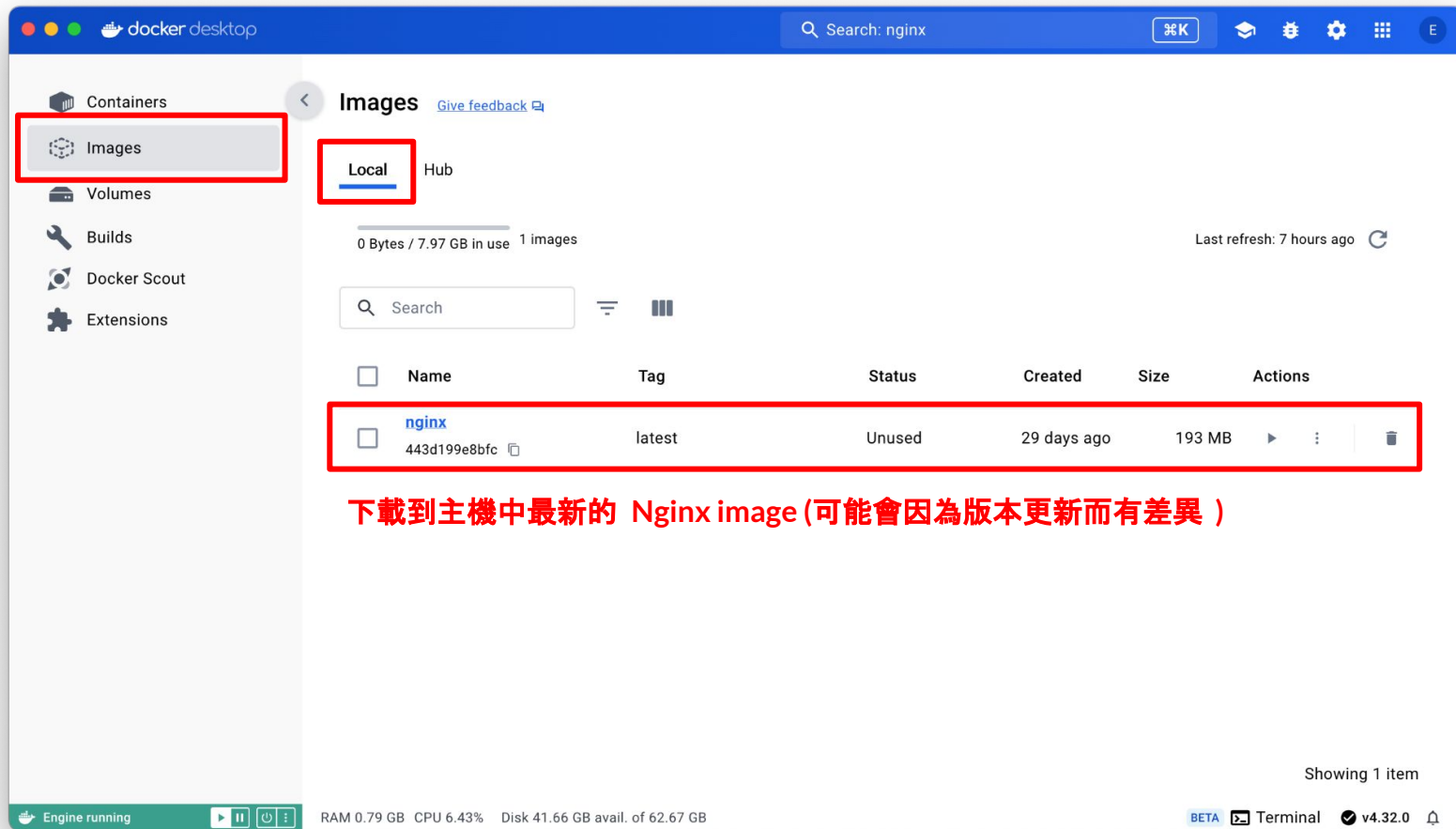
透過 Docker Desktop 在電腦主機上建立並執行一個提供 Nginx 網頁伺服器的 container。

備註: Nginx 是一個開放原始碼的網頁伺服器應用程式。

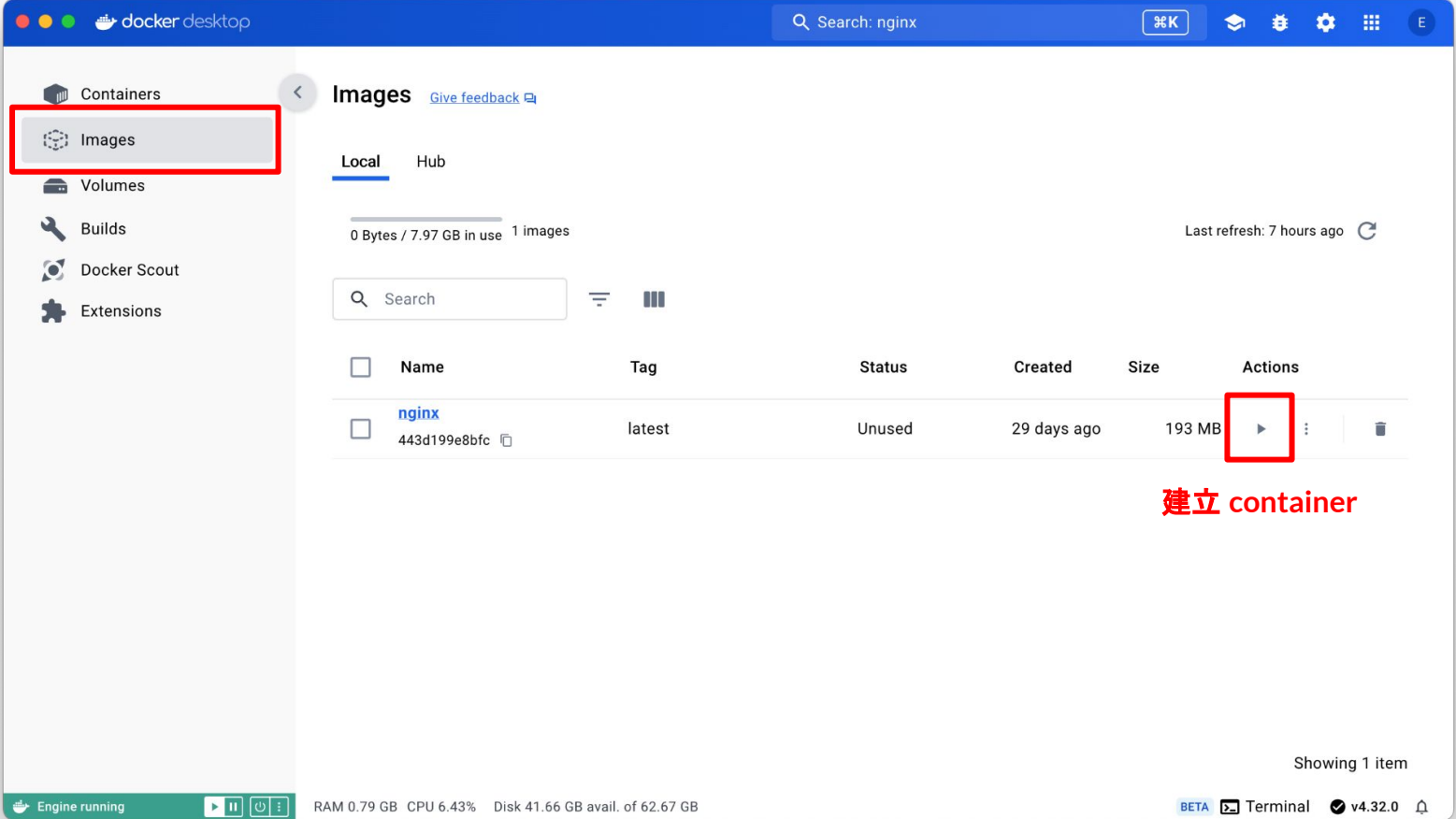






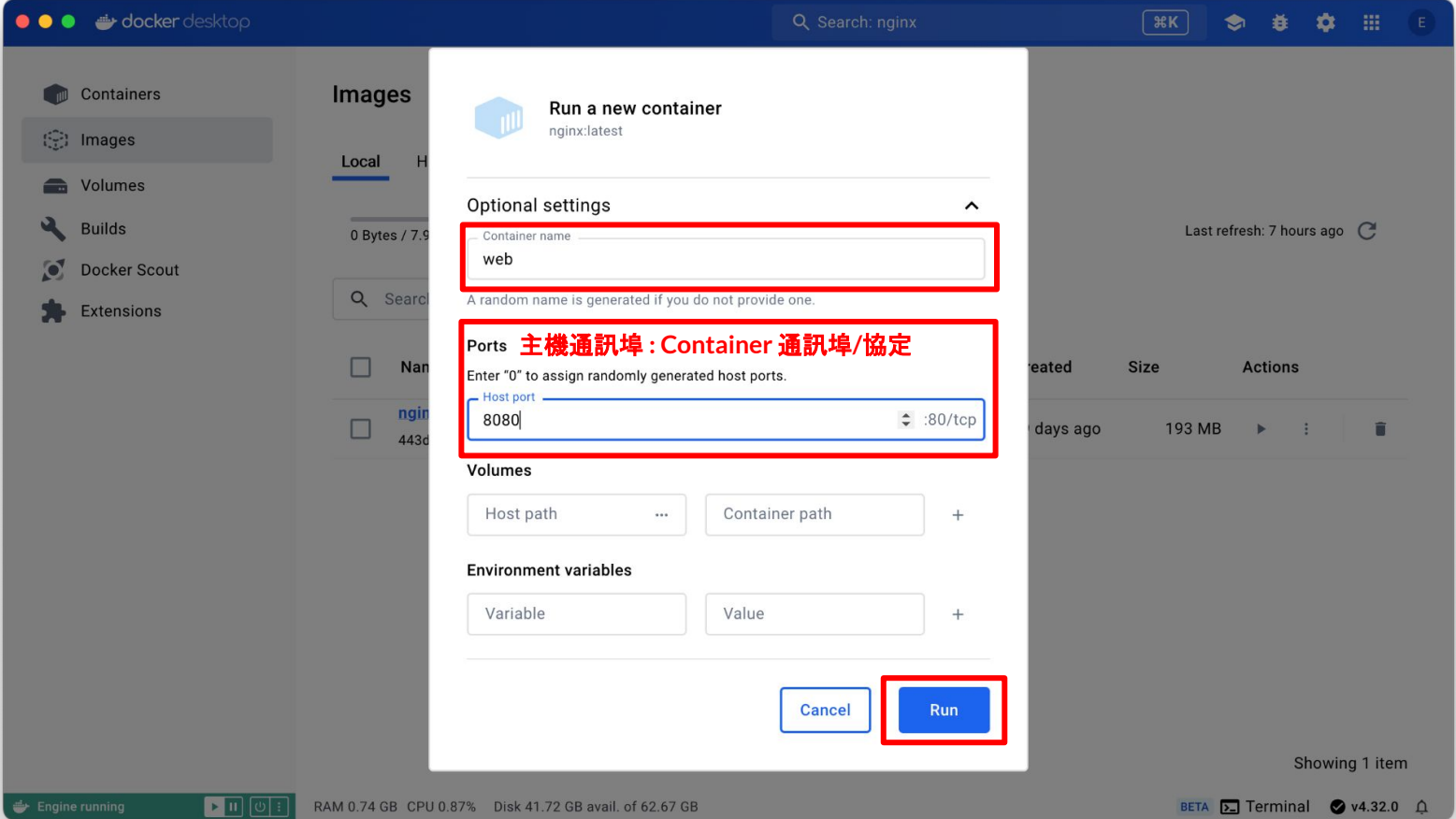


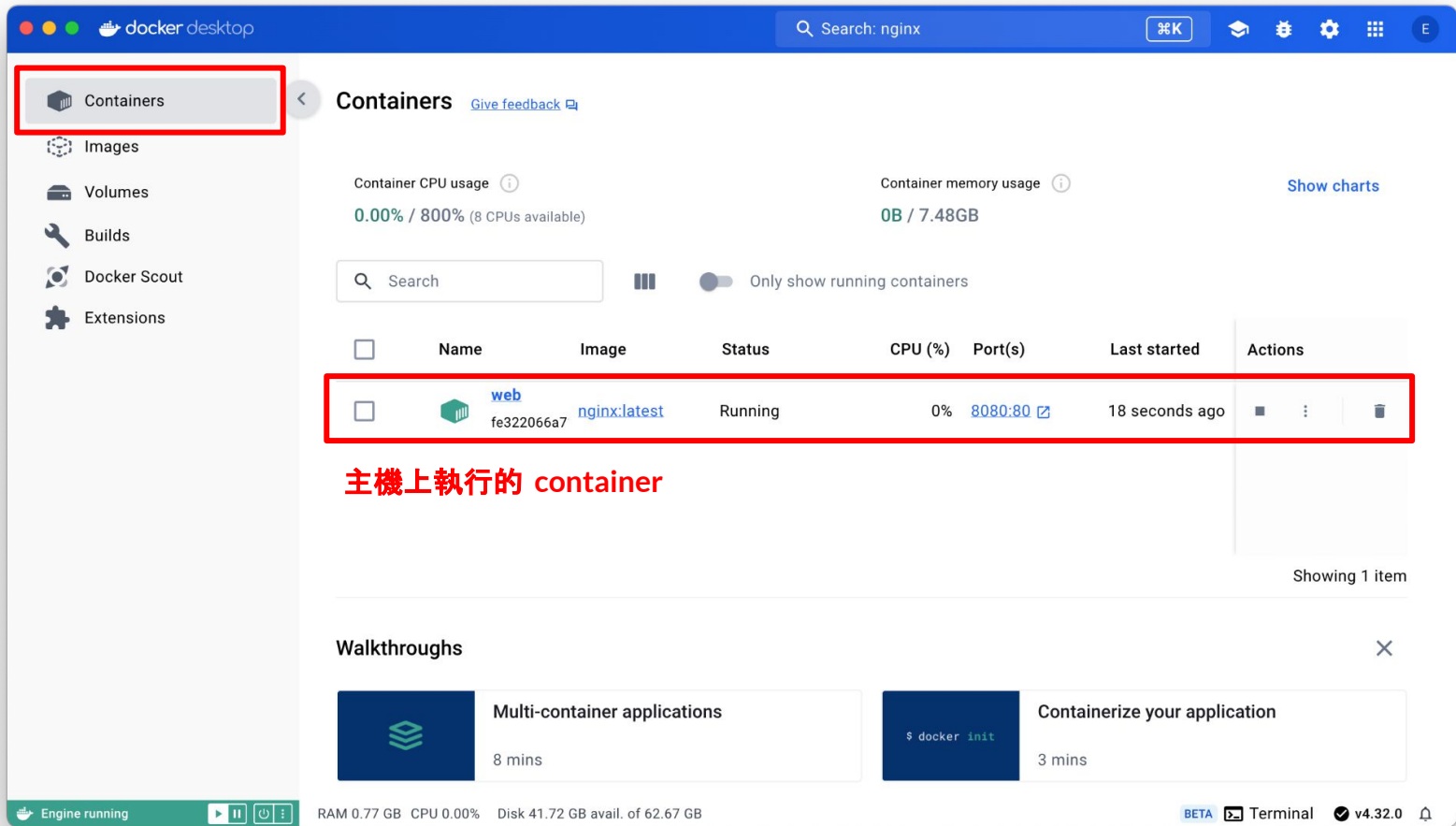
下載到主機中最新的 Nginx image (可能會因為版本更新而有差異)



建立 container

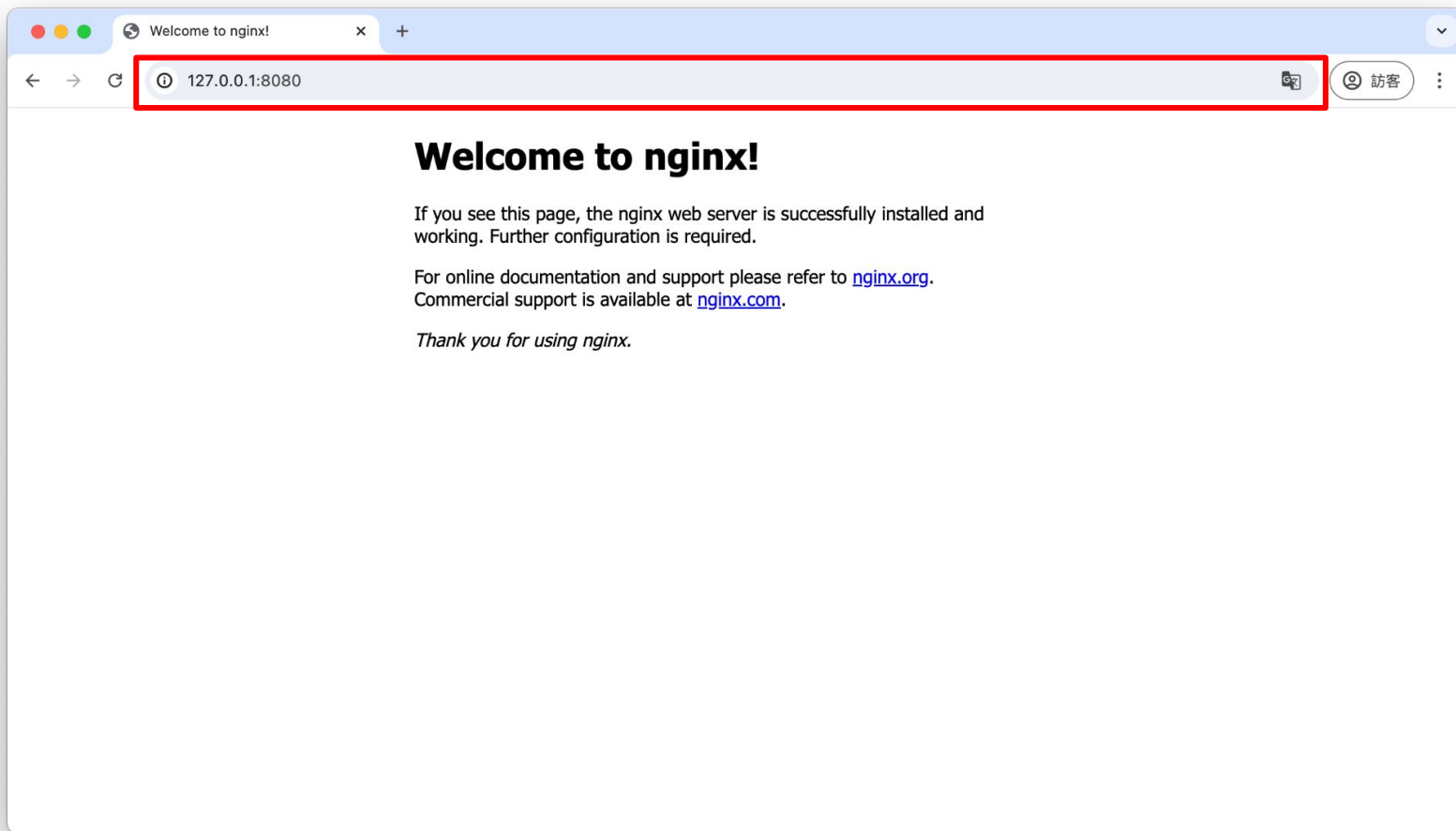


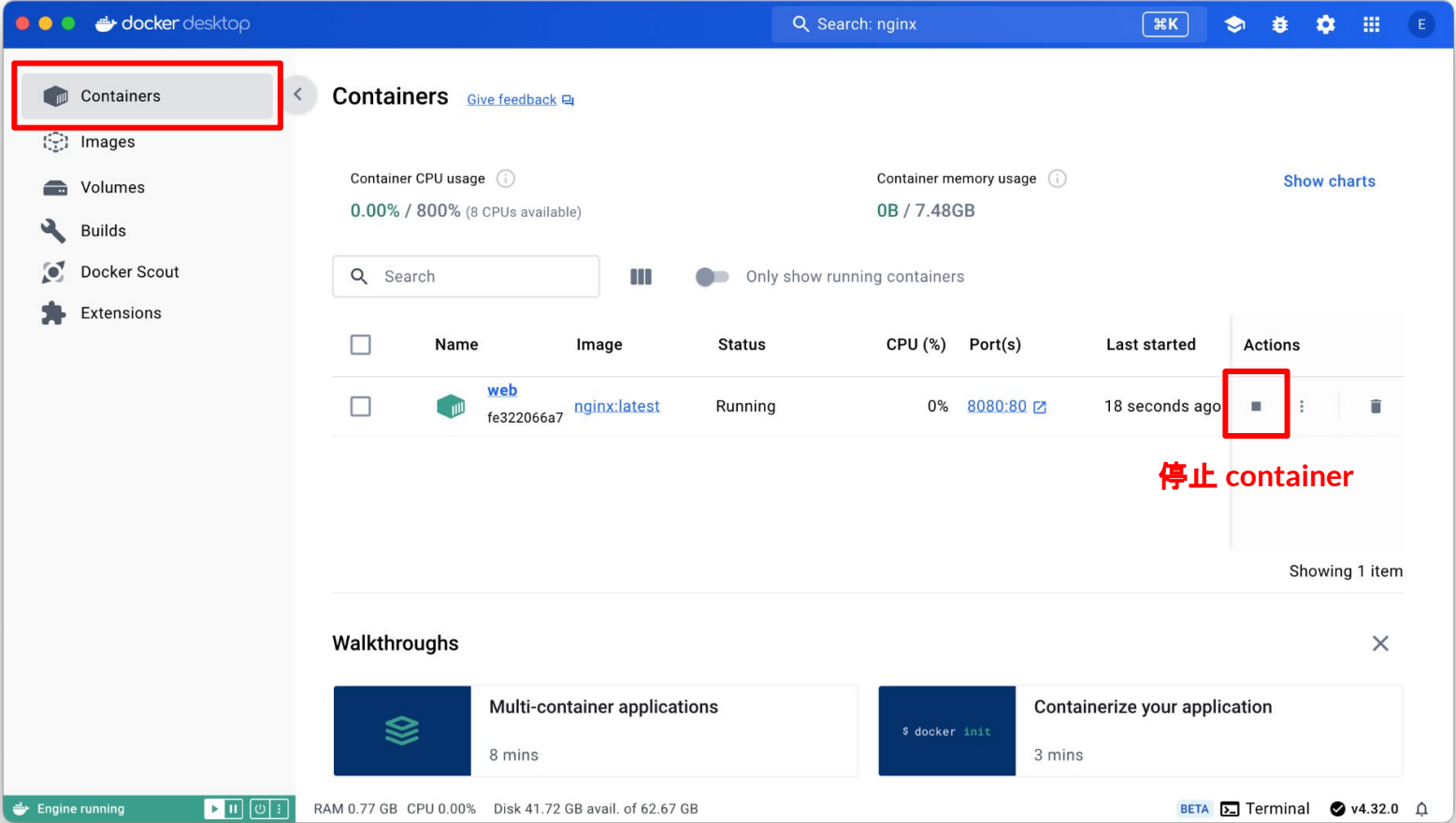




主機上執行的 container

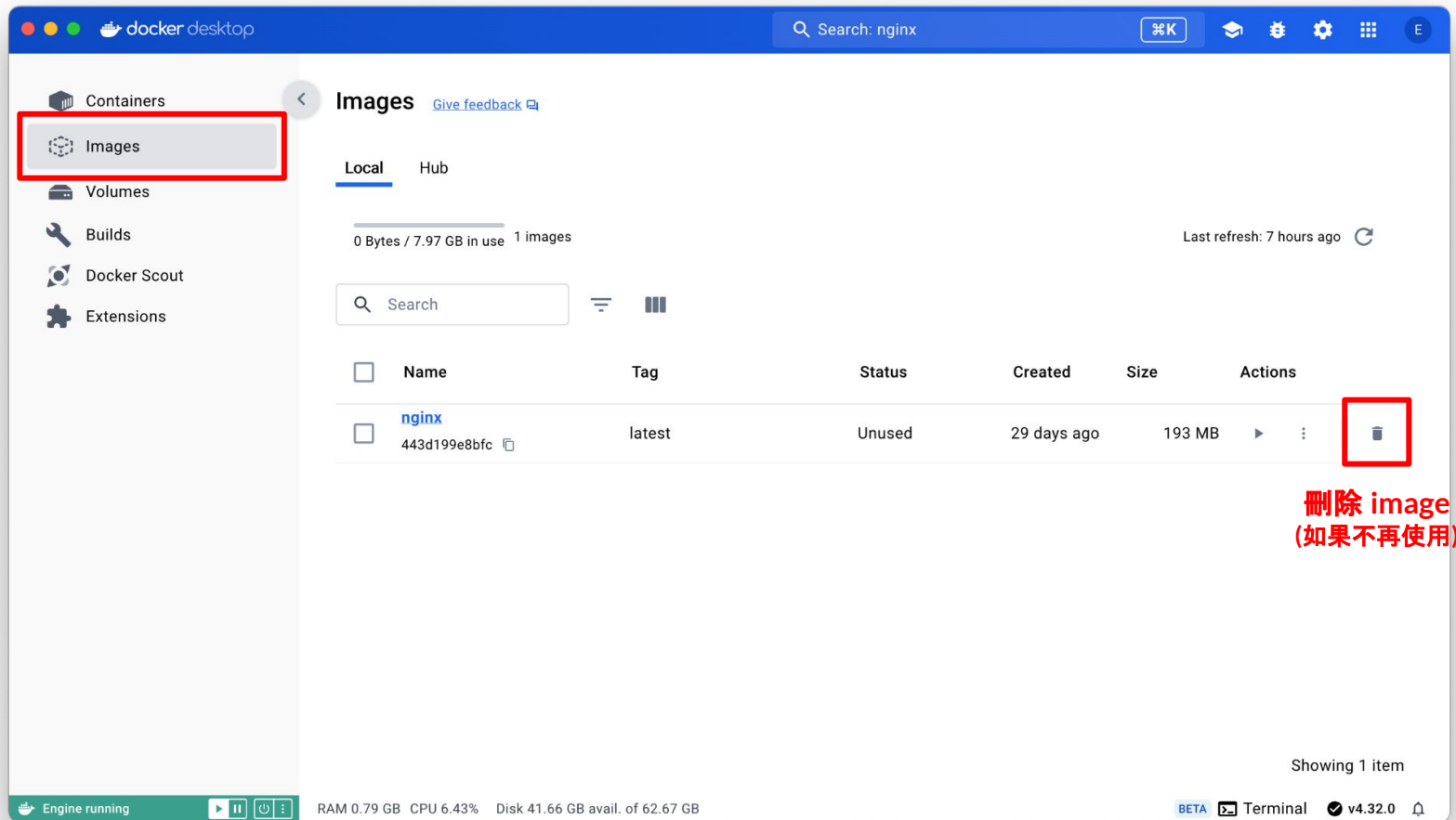








刪除執行環境



使用案例 #2: Nginx (與主機共用資料夾)

需求情境

已經在電腦主機上架設好 Nginx 網頁伺服器了，但是需要置換成自己設計的網站內容。

操作步驟

透過 Docker Desktop 在電腦主機上執行 Nginx 的 container 並將主機的網站資料夾對應到 container 的網站資料夾 (/usr/share/nginx/html)。

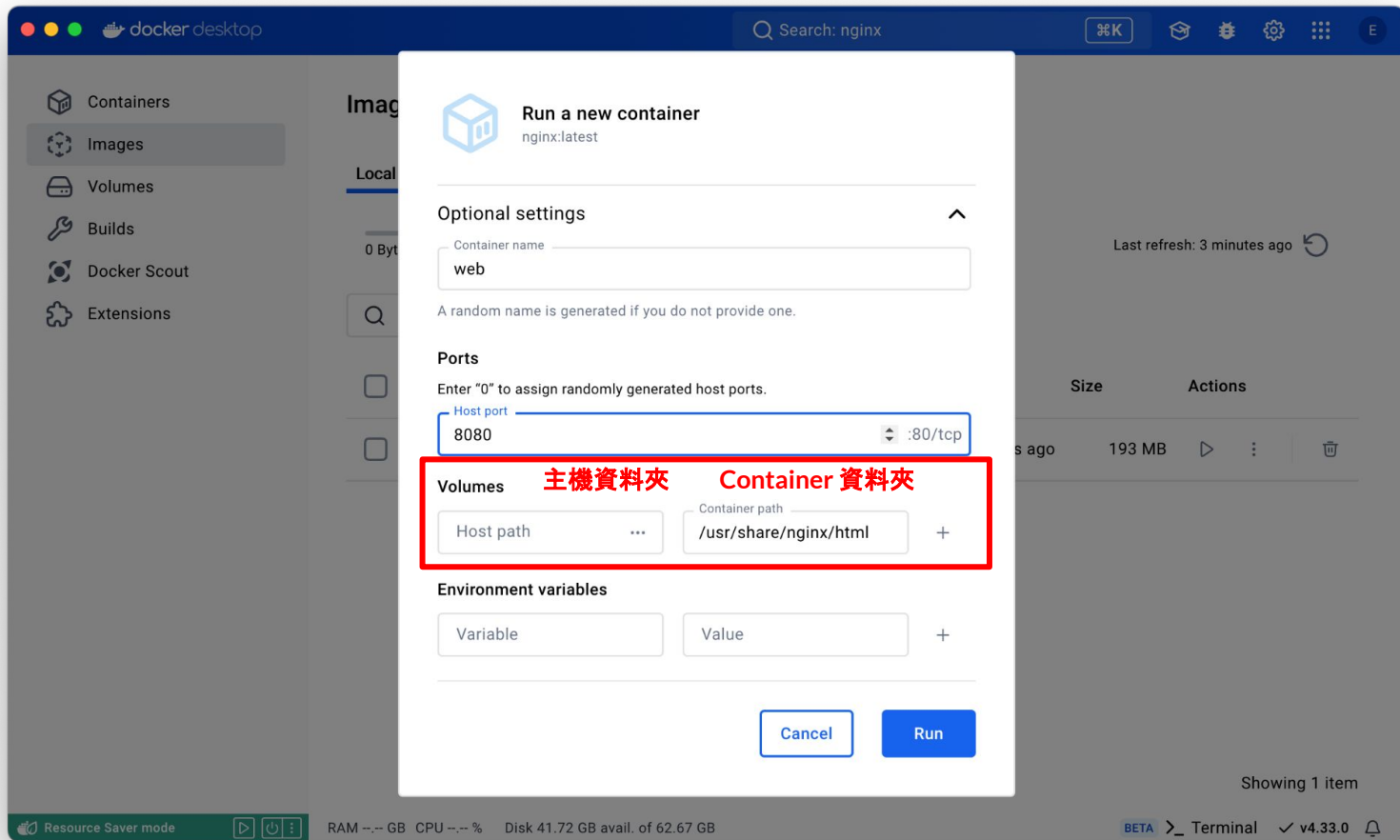


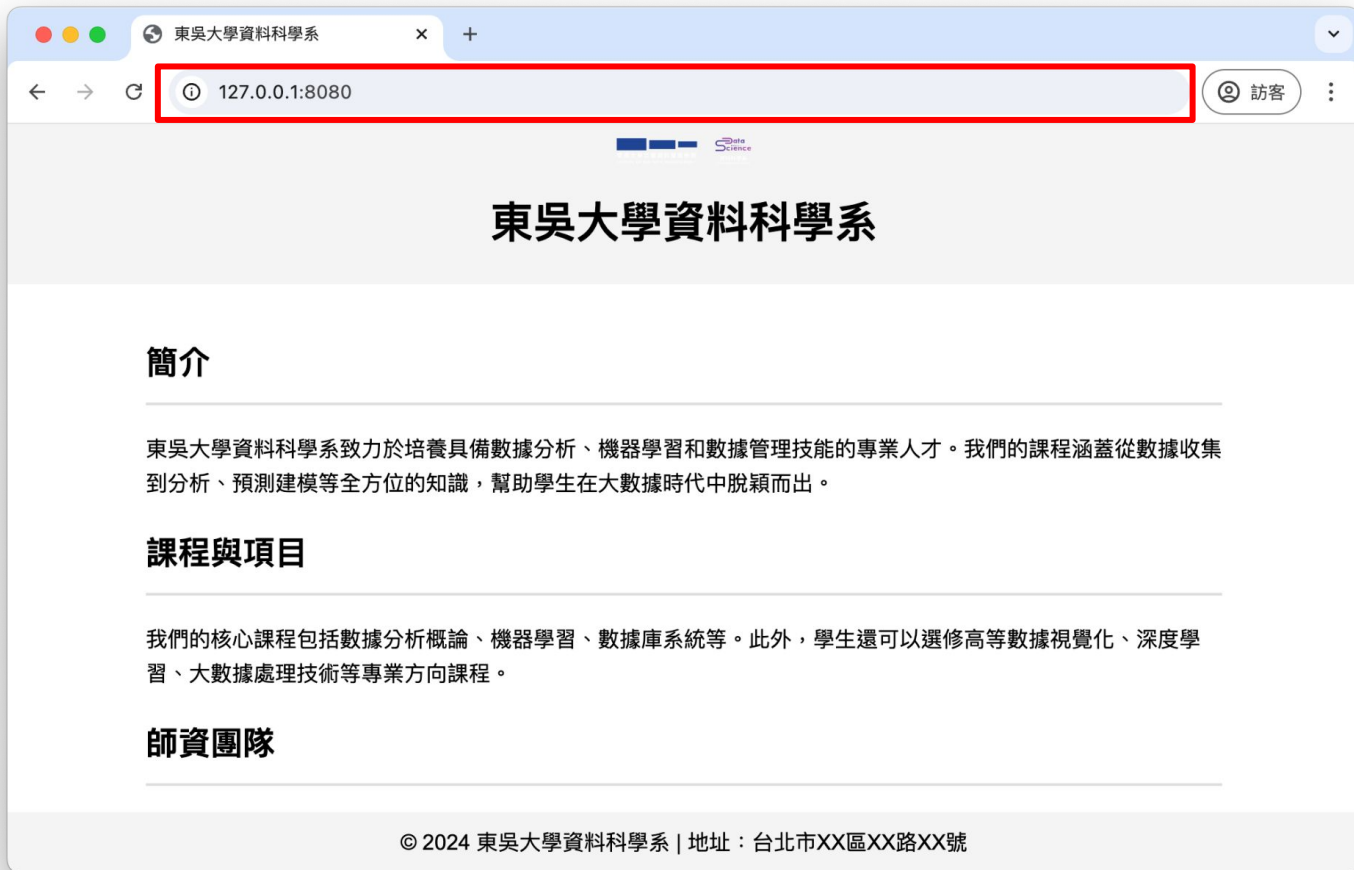
在主機上準備好網站 內容

如果沒有自己的網站內容，可以直接使用老師提供網頁[\[點此下載\]](#)。

下載 scu_web.zip 檔案後，在本機上解壓縮後產生 scu_web 資料夾。









Hands-on Lab#2

使用 Docker Desktop 在自己的電腦主機上建立一個 Jupyter Notebook 開發環境

難度: 中等

時間: 15分鐘

練習目標

透過 Docker Desktop 在主機上建立並執行一個提供 [Jupyter Notebook](#) 開發環境的 container。

操作步驟

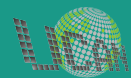
1. 建立執行 Jupyter Notebook 的 container
Image: [jupyter/datascience-notebook](#)
2. 需要掛載本機資料夾
3. 確認 Jupyter Notebook 可正常運作
4. 刪除執行 Jupyter Notebook 的 container

備註1: Jupyter Notebook 是一個基於網頁服務的互動式 Python 程式開發平台, 也是學習資料科學的常見工具之一。

備註2: 預設的工作目錄為 `/home/jovyan`, 所以可以試著對應到主機資料夾後取出專案檔案。

使用 Docker CLI

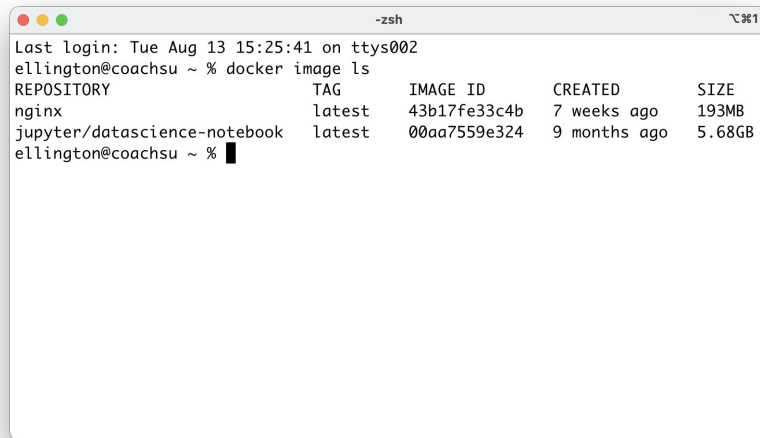
為何使用Docker CLI?



為何使用 Docker CLI?

使用 Docker Command Line Interface (CLI) 的原因很多, 例如

- **進階的控制**: Docker CLI 提供了更多的控制選項, 因此能夠更靈活使用。
- **環境的限制**: 某些特定環境(如雲端環境)只支援 CLI 介面而不提供桌面環境。
- **自動化需求**: 使用 Docker CLI 才能夠實現自動化流程。
- ...

A terminal window titled '-zsh' showing the output of the 'docker image ls' command. The output is a table with columns: REPOSITORY, TAG, IMAGE ID, CREATED, and SIZE. It lists two images: 'nginx:latest' and 'jupyter/datascience-notebook:latest'.

```
-zsh
Last login: Tue Aug 13 15:25:41 on ttys002
ellington@coachs ~ % docker image ls
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
nginx                latest      43b17fe33c4b  7 weeks ago   193MB
jupyter/datascience-notebook  latest     00aa7559e324  9 months ago  5.68GB
ellington@coachs ~ %
```





Docker CLI 指令格式

開啟終端機中執行 Docker CLI 指令，指令格式如下

```
$ docker [OPTIONS] <COMMAND>
```

例如，執行下列指令可以取得 Docker CLI 命令的使用說明

```
$ docker --help
```

```
$ docker <COMMAND> --help
```

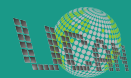
備註1: 指令中 `[]` 代表非必要的部分；`<>` 代表必要但可以根據需求而改變的部分

備註2: Windows 作業系統建議從 Microsoft Store 中安裝 Windows Terminal 來使用



使用 Docker CLI

基本Image管理





常用的 Image 管理命令

從 registry 下載 image (預設的 registry 為 Docker Hub)

```
$ docker image pull <IMAGE URI>
```

列出本機中所有 images

```
$ docker image ls
```

刪除本機 image (前提是本機中不存在任何基於此 image 建立的 container)

```
$ docker image rm <IMAGE URI|ID>
```

備註: 指令中 | 代表 "或"





使用 Docker CLI 管理 Image (課堂練習)

從 registry 下載 image (如果沒有加 tag, 預設為 latest)

```
$ docker image pull nginx
```

```
$ docker image pull public.ecr.aws/nginx/nginx
```

列出本機中所有的images

```
$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	43b17fe33c4b	7 weeks ago	193MB
public.ecr.aws/nginx/nginx	latest	43b17fe33c4b	7 weeks ago	193MB



使用 Docker CLI 管理 Image (課堂練習)

刪除本機 image

```
$ docker image rm public.ecr.aws/nginx/nginx
```

列出本機中所有的images

```
$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	43b17fe33c4b	7 weeks ago	193MB



Hands-on Lab#3

使用 Docker CLI 管理 Image

難度: 簡單

時間: 10分鐘

練習目標

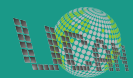
練習使用 Docker CLI 管理 image。

操作步驟

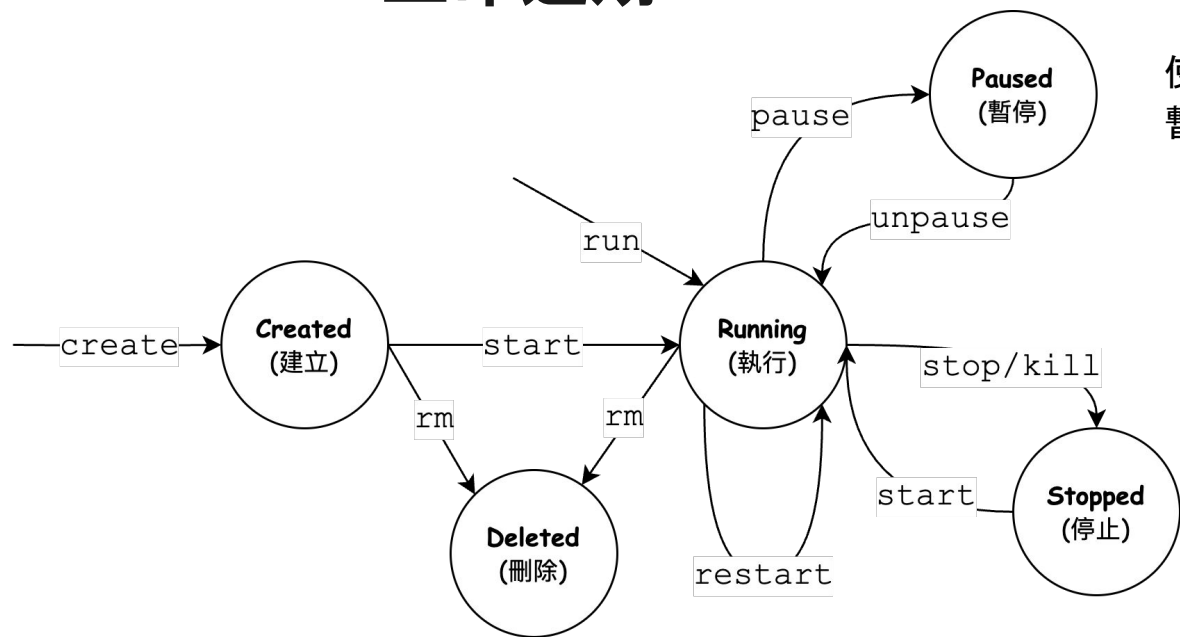
1. 使用 Docker CLI 下載 image
Image: [jupyter/datascience-notebook](https://hub.docker.com/r/jupyter/datascience-notebook)
2. 使用 Docker CLI 確認已下載的 image
3. 使用 Docker CLI 刪除已下載的 image

使用 Docker CLI

基本 Container 管理



Container 生命週期



使用 `pause` 會發出 `SIGSTOP` 暫停所有程序。

- 使用 `stop` 會先發出 `SIGTERM` 訊號讓服務有時間正常停止。**(建議使用)**
- 使用 `kill` 會直接發出 `SIGKILL` 訊號停止服務。



常用的 Container 管理命令

建立並執行一個 container

```
$ docker container run [OPTIONS] <IMAGE URI|URI>
```

常用選項

<code>-d</code>	背景執行container並回傳ID
<code>-i</code>	開啟標準輸入(STDIN)
<code>-t</code>	執行時配置一個終端機
<code>--name <CONTAINER NAME></code>	設定container名稱
<code>-p <HOST PORT:CONTAINER PORT></code>	設定主機與container的通訊埠對應
<code>-v <HOST DIR:CONTAINER DIR></code>	設定主機與container的資料夾對應





常用的 Container 管理命令(續)

列出本機中的container

```
$ docker container ls [OPTIONS]
```

常用選項

- a 顯示所有container (預設只會顯示執行中的container)
- s 顯示container已使用的磁碟空間



常用的Container管理命令(續)

刪除本機 container

```
$ docker container rm [OPTIONS] <CONTAINER NAME|ID>
```

常用選項

-f 強制刪除執行中的container

備註: 預設無法刪除執行中的container





使用 Docker CLI 管理 Container (課堂練習)

開啟第一個終端機建立並執行一個可以透過終端機互動的ubuntu linux 環境

```
$ docker image pull ubuntu  
$ docker container run -it --name linux1 ubuntu  
root@<CONTAINER ID>:/#
```

停止第一個ubuntu linux 後建立並執行另一個ubuntu linux 環境

```
root@<CONTAINER ID>:/# exit  
$ docker container run -it --name linux2 ubuntu  
root@<CONTAINER ID>:/#
```



使用 Docker CLI 管理 Container (課堂練習)

開啟第二個終端機列出目前正在執行的ubuntu linux

```
$ docker container ls
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
aa10c97373ce	ubuntu	"/bin/bash"	3 minutes ago	Up 3 minutes		linux2

列出所有的ubuntu linux

```
$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
aa10c97373ce	ubuntu	"/bin/bash"	3 minutes ago	Up 3 minutes		linux2
cfa73d0c7ebd	ubuntu	"/bin/bash"	3 minutes ago	Exitd (0) ..		linux1



使用 Docker CLI 管理 Container (課堂練習)

在第二個終端機刪除已停止的ubuntu linux

```
$ docker container rm linux1
```

```
$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
aa10c97373ce	ubuntu	"/bin/bash"	5 minutes ago	Up 5 minutes		linux2



其它有用的指令

停止正在执行的container

```
$ docker container stop <CONTAINER NAME|ID>
```

重新执行停止中的container

```
$ docker container start <CONTAINER NAME|ID>
```

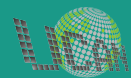
在执行中的container 上执行命令

```
$ docker container exec [OPTIONS] <CONTAINER NAME|ID> <COMMAND>
```



使用 Docker CLI

使用案例



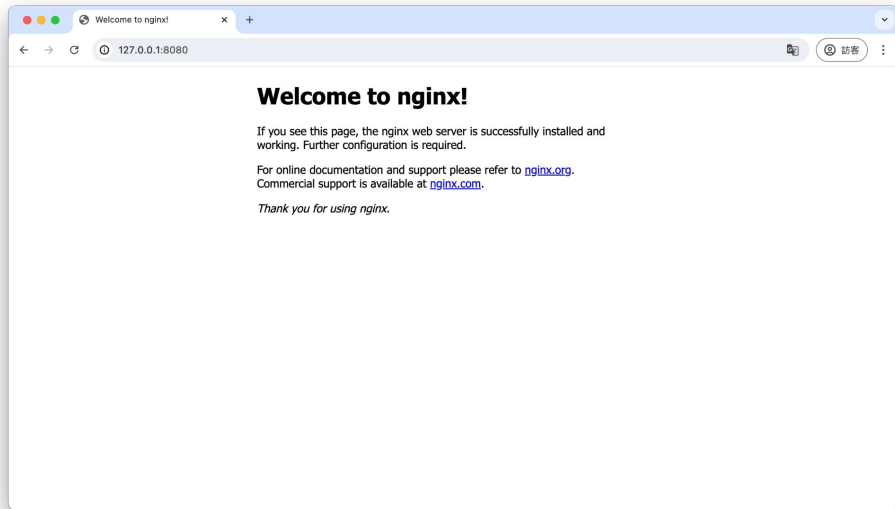
使用案例 #1: Nginx 執行環境

需求情境

專題(或作業)需要在電腦主機上架設一個網頁伺服器。

操作步驟

透過 Docker CLI 在電腦主機上建立並執行一個提供 Nginx 網頁伺服器的 container。





建立執行環境

開啟終端機建立並執行一個提供Nginx 網頁伺服器的container

```
$ docker image pull nginx
```

```
$ docker container run -d -p 8080:80 --name web nginx
```

刪除執行環境

```
$ docker container stop web
```

```
$ docker container rm web
```



使用案例 #2: Nginx (與主機共用資料夾)

需求情境

已經在電腦主機上架設好 Nginx 網頁伺服器了，但是需要置換成自己設計的網站內容。

操作步驟

透過 Docker CLI 在電腦主機上執行 Nginx 的 container 並將主機的網站資料夾對應到 container 的網站資料夾 (/usr/share/nginx/html)。





建立執行環境

開啟終端機建立並執行一個提供Nginx 網頁伺服器的container

```
$ docker container run -d -p 8080:80 -v <HOST PATH>:/usr/share/nginx/html  
---name web nginx
```

刪除執行環境

```
$ docker container stop web  
$ docker container rm web
```

備註: 如果主機為Windows則資料夾"C:\Users\scu\web"的表示方式為"/C/Users/scu/web"





Hands-on Lab#4

使用 Docker CLI 建立一個 Jupyter Notebook 服務

難度: 困難

時間: 25分鐘

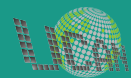
練習目標

透過 Docker CLI 在主機上建立並執行一個提供 [Jupyter Notebook](#) 環境的 container。

操作步驟

1. 建立執行 Jupyter Notebook 的 container
Image: [jupyter/datascience-notebook](#)
2. 需要掛載本機資料夾
3. 確認 Jupyter Notebook 可正常運作
4. 刪除執行 Jupyter Notebook 的 container

製作 Docker 映像檔





產生 Image

當需要客製化映像檔時，可以從現有的 Image (稱為parent Image)去產生新的客製化 Image。例如

- 利用 Container 產生新的 Image
 - 先將 Container 客製化後產生成 Image
- 利用 Dockerfile 產生新的 Image
 - 撰寫 Dockerfile 並透過指令產生 Image

方法一

利用 Container 產生新的 Image



Step 1. 用 Parent Image 產生 Container

例如，下載 ubuntu Image 作為 Parent Image

```
$ docker image pull ubuntu
```

建立並執行可以終端機互動的 Container

```
$ docker container run -it --name myubuntu ubuntu  
root@<id>:/#
```





Step 2. 在 Container 上安裝需要的套件

安裝套件前須先更新軟體資料庫

```
root@<id>:/# apt update
```

安裝需要的套件

```
root@<id>:/# apt install nginx
```

...



Step 3. 從 Container 產生新的 Image

利用 commit 命令以 Container 目前的狀態產生新的 Image。格式如下

```
$ docker container commit <container name or id> <new Image URI>
```

其中,

1. 加入 -m 選項可以添加 Image 說明
2. 加入 -a 選項可以添加作者資訊 (格式如 W.-T. Su <suwt@au.edu.tw>)



Step 3. 從 Container 產生新 Image (範例)

利用commit命令以Container目前的狀態產生新的Image

```
$ docker commit myubuntu ellington/ubuntu:nginx
```

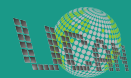
確認產生的新Image

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ellington/ubuntu	nginx	bbd063ddc967	23 hours ago	375MB
ubuntu	latest	452a96d81c30	4 weeks ago	79.6MB

方法二

利用 Dockerfile 產生新 Image





Step 1. 撰寫 Dockerfile

Dockerfile 可以透過批次命令將從 Parent Image 到產生新 Image 中間的步驟自動化。Dockerfile 的格式為

`INSTRUCTION arguments`

(例如: `FROM ubuntu` 的意思是以 ubuntu 為 Parent Image)

Dockerfile 支援的 `INSTRUCTION` 非常多, 請參考 [Dockerfile文件](#)。





Step 1. 撰寫 Dockerfile (續)

這裡以安裝網站伺服器為例, Dockerfile 內容如下

```
# FROM指令用來指定 Parent Image 為 ubuntu
FROM ubuntu
# RUN指令可以在以 Parent Image 產生的 Container 中執行命令並寫回新 Image
RUN apt update
RUN apt install -y nginx
```



Step 2. 從 Dockerfile 產生新 Image

利用 build 命令可以依據 Dockerfile 產生新 Image。格式如下

```
$ docker image build -t <new Image URI> <path to Dockerfile>
```

其中,

1. 加入 `-t` 選項可以幫新 Image 命名(雖然不是必要, 但建議使用)

Step 2. 從 Dockerfile 產生新 Image (範例)

例如，從剛剛撰寫的 Dockerfile 產生新 Image

```
$ docker build -t ellington/ubuntu:nginx .
```

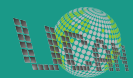
確認新產生的映像檔

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ellington/ubuntu	nginx	0d27c8edfbc	1 hours ago	375MB
ubuntu	latest	452a96d81c30	4 weeks ago	79.6MB



分享 Docker 映像檔





方法一：匯出 / 匯入 Image

匯出 image

```
$ docker save -o image.tar <image name>
```

將image.tar分享給需要的人

匯入 image

```
$ docker load -i image.tar
```




方法二: 分享 Dockerfile

將 Dockerfile 分享給需要的人

利用 build 命令產生新映像檔。

```
$ docker build -t <image name> <path to Dockerfile>
```

其中,

1. 加入 `-t` 選項可以幫新映像檔命名 (雖然不是必要, 但建議使用)
2. `<image name>` 遵照 `[<user name>/]<repo name>[:<tag name>]` 格式



方法三: 分享至 Docker Hub

登入 Docker 並輸入帳號/密碼(請先到 <http://hub.docker.com> 註冊)

```
$ docker login
```

將想要分享的image 重新命名(如果有必要)

```
$ docker tag <old image id> <new image name>
```

(新映像檔命名格式為:<user name>/<repo name>:<tag>)

將 image 上傳至 Docker Hub

```
$ docker push <new image name>
```



Hands-on Lab

產生/分享 Image

時間:30 分鐘

練習產生/分享 Image

1. 製作 Image 如下
 - 以ubuntu為基礎
 - 安裝nginx (預設網頁放在 /var/www/html)
 - 將網頁改成Hello, World!
2. 將 Image 上傳至 DockerHub
3. 刪除本地 Image
4. 從 DockerHub下載上傳的 Image
5. 建立 Container 並執行 nginx 服務
6. 以瀏覽器確認nginx服務正常運作

Q & A



Computer History Museum, Mt. View, CA